서울시립대학교 · 전자전기컴퓨터설계실험II

LAB7 실험 후 보고서

STM32_3 — SPI로 MCU와 FPGA 연결 (과제8 MCU 송신 파형 · 과제9 MCU↔MCU · 과제10 MCU→FPGA · 과제11 FPGA→MCU · 과제12 FPGA↔FPGA)

이름
형종희

학번
2023440145

조
C조 (과제9·12 짝: D조)

실험일
2026-11-02

작성일
2026-11-02

FPGA 보드
Combo II-DLD S75 · xc7s75fgga484-1

MCU 보드
NUCLEO-G474RE

측정 장비
Tektronix TBS2102B (100 MHz, 2 GS/s)

📦 GitHub 제출 정보

항목	값
저장소	https://github.com/hyeongjonghee/electronics2-LABs (Public)
실험 전 보고서	태그 `lab7-prelab` = `43e739e` (설계 소스 커밋 `68e6369`)
실험 당일 기록 커밋	`c78deab` — 영상 8개(`LAB7/evidence/videos/*.mp4`)와 사진 6장
LAB7 경로	`LAB7/circuits/LAB7-01~03/`, `LAB7/stm32/`, `LAB7/evidence/`, `LAB7/LAB7_실험후보고서.html` · `.pdf`
최종 제출 태그	`lab7-submit` — 이 보고서(HTML · PDF)가 들어간 커밋
STM32 기반 프로젝트	`Glaysia/ece2_474` v1.1.0 = `dd94f01` (10/30 이후 수업 저장소는 `Glaysia/ece2_26_2_2`로 통합됨. STM32_3 문서 내용은 같음을 확인)

0. 결과 요약

11/2 실습에서 STM32_3의 과제 5개를 모두 보드에서 동작시켰다. 진행 순서는 짝 조(D조)를 기다리지 않도록 **우리 조 장비만 쓰는 과제를 먼저** 했다. 과제8 → 과제10 → 과제11 → 과제9 → 과제12 순서다. 실험 전에 만든 비트스트림 2개와 MCU 코드 3개를 **다시 빌드하지 않고 그대로** 썼고, 교재 p.73 핀표와도 일치했다.

#	실험 (무엇을 하는가)	함께	측정 · 관찰	결과
1	**과제8** MCU 송신 파형 — MCU가 1초마다 보내는 SPI 바이트의 SCK·MOSI를 오실로스코프로 보고 비트 순서와 클록 주파수를 잰다	혼자	SCK 펄스 8개, 상승 에지에서 읽은 MOSI = 0x55 · 0xA5 · 0x00 · 0xFF, **777.7 ~ 781.0 kHz**	**계산값 781.25 kHz와 −0.45 ~ −0.03 % ✔**
2	**과제10** MCU → FPGA — MCU가 보낸 바이트를 직접 만든 FPGA 수신 회로가 받아 FPGA USB-UART로 PC에 "RX=xx" 출력	우리 조	FPGA COM: `RX=00 · FF · 55 · A5 · 00`, LED 8개 같은 값	**MCU 송신 순서와 일치 ✔**
3	**과제11** FPGA → MCU — 직접 만든 FPGA 송신 회로가 1초마다 DIP 스위치 값을 보내고 MCU(슬레이브)가 받아 출력	우리 조	SCK **498.7 ~ 502.6 kHz**, MCU `RX=55` ×10, 스위치를 바꾸면 1초 안에 MCU 값이 바뀜	**설계값 500 kHz ✔**
4	**과제9** MCU ↔ MCU — 두 조의 MCU를 연결해 한쪽이 보내고 다른 쪽이 받는다, 역할 교대	D조	양방향 모두 `RX=55 · A5 · 00 · FF` 반복	**양방향 ✔**
5	**과제12** FPGA ↔ FPGA — 한쪽 송신 회로, 다른 쪽 수신 회로로 FPGA끼리 통신, 역할 교대	D조	1차 D조 수신 `55 55 …` (Hex), 2차 우리 수신 `RX=FF · 55 · A5 · 00`	**2차 처음 글자 깨짐 → 클록 다이얼 고친 뒤 ✔**

실험 전 보고서 대비 바뀐 점

- **설계는 바뀐 것이 없다.** FPGA 수신·송신 회로, XDC 핀(F8 · E8 · A7), MCU 코드 모두 실험 전 그대로 동작했다.
- **실험용 노트북이 바뀌었다**(대여 노트북 반납). 그래서 COM 번호가 달라졌다: NUCLEO = **COM3**, FPGA USB-UART = **COM5**. 비트스트림은 이 노트북에서 다시 빌드해 실험 전과 같은 자원·타이밍(LUT 160 / 207, WNS +14.907 / +14.323 ns)을 확인했고, Vivado를 한 번만 켜 두고 다운로드하는 방식으로 한 번 올리는 데 약 10 s가 걸렸다.
- **과제12 상대 회로**: D조 수신 회로는 받은 바이트를 글자로 바꾸지 않고 그대로 UART로 보낸다. 그래서 0x55가 Serial Monitor Text 모드에서 `U`로 보였고, Hex 모드로 바꿔 `55`로 확인했다(5장).

목차

1. 과제8 · MCU 송신 파형
2. 과제10 · MCU → FPGA
3. 과제11 · FPGA → MCU
4. 과제9 · MCU ↔ MCU
5. 과제12 · FPGA ↔ FPGA
6. 교재 핀 대조 · 보드 빌드
7. 고찰
8. 참고문헌

공통 측정 환경

- **SPI 규칙**(수업 통일안): 모드 0 — SCK 대기 LOW, 수신 측은 상승 에지에서 MOSI를 읽고 송신 측은 하강 에지에서 바꾼다. 8비트, MSB 먼저, CS LOW 동안만 통신, MISO 없음.
- **배선**: NUCLEO Arduino 헤더 D3(PB3, SCK) · D11(PA7, MOSI) · D10(PB6, CS 출력) · A2(PA4, 슬레이브 NSS 입력). FPGA는 확장 포트 PORT A 헤더 J10의 **1번(F8, SCK) · 3번(E8, MOSI) · 5번(A7, CS)**. GND는 항상 공통.
- **FPGA 보드 설정**: CLOCK SELECT = F (50 MHz), PORT A ENABLE(DIP SW5의 2번) = OFF.
- **PC 출력**: MCU는 ST-LINK 가상 COM(COM3), FPGA는 FTDI USB-UART(COM5), 모두 9600 bps 8N1. VS Code Serial Monitor 또는 PowerShell `listen7`로 읽었다.

1. STM32_3 과제8 — MCU 송신 파형 확인

MCU 한 대가 SPI로 1초마다 한 바이트를 보내고, 오실로스코프로 SCK와 MOSI를 보며 실제 비트 순서와 클록 주파수를 기록한다.

① 목적

SPI가 실제 선 위에서 어떻게 생겼는지 직접 본다. 박자(SCK)를 치는 사람이 마스터이고, 박이 올라갈 때마다 받는 쪽이 글자 한 칸(MOSI)을 읽는다. 0x55 · 0xA5 · 0x00 · 0xFF를 보내 비트가 MSB부터 나오는지, SCK가 계산값과 같은지 확인한다.

② 이론

$$f_{SCK} = \frac{100\ \text{MHz}}{128} = 781.25\ \text{kHz}, \qquad T_{SCK} = 1.28\ \mu\text{s}, \qquad 8\,T_{SCK} = 10.24\ \mu\text{s}$$

한 바이트(약 10 µs)가 1초에 한 번만 지나가므로, 화면을 계속 그리는 자동(Auto) 트리거로는 거의 잡히지 않는다. **트리거 모드 일반(Normal) + Single**로 SCK 상승 에지를 기다렸다가 한 번 잡는다.

③ 코드

매뉴얼 송신 예제에서 보내는 값만 1초마다 55 → A5 → 00 → FF로 바꾼 판(`LAB7/stm32/task8_cycle`)을 썼다. 다시 빌드하지 않고 네 값을 모두 보기 위해서다. 설명은 실험 전 보고서 1장.

④ Vivado 시뮬레이션 (실험 전 예측)

XSim으로 MCU 모델(781.25 kHz, 모드 0)이 0x55를 보내는 파형을 미리 그려 두었다. 위 세 줄이 오실로스코프에 보일 신호이고, 빨간 점선(상승 에지)에서의 MOSI가 0 1 0 1 0 1 0 1이다. 아래 그림 1의 실측과 모양이 같다.

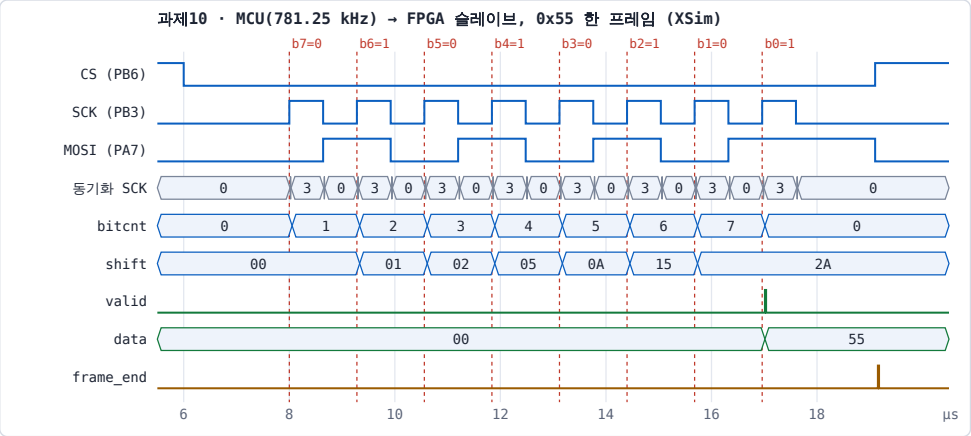


그림 S1. XSim — MCU 송신 0x55 한 프레임과 이를 받는 FPGA 내부(과제10). 실험 전 보고서 그림 1과 같음.

⑤ 보드 검증

설정: CH1 = SCK(D3), CH2 = MOSI(D11), 둘 다 1 V/칸 · DC, 2 µs/칸, 트리거 CH1 상승 1.52 V · Normal · Single. 상승 에지(노란 펄스의 올라가는 순간)마다 파란 선의 높이를 읽었다.

보낸 값	상승 에지에서 읽은 MOSI (b7 → b0)	SCK 측정	오차	판정
0x55	0 1 0 1 0 1 0 1	780.6 kHz	−0.08 %	✔
0xA5	1 0 1 0 0 1 0 1 (가운데 b4·b3 연속 0)	777.7 · 779.3 · 780.8 kHz	−0.45 ~ −0.06 %	✔
0x00	0 0 0 0 0 0 0 0 (MOSI 계속 LOW)	781.0 kHz	−0.03 %	✔
0xFF	1 1 1 1 1 1 1 1 (MOSI 계속 HIGH)	777.7 kHz	−0.45 %	✔

측정값은 모두 계산값보다 조금 작다(최대 0.45 %). 펄스가 8개뿐인 짧은 구간에서 오실로스코프가 주기를 평균 내기 때문에 생기는 측정 오차 범위다. MCU 클록 자체(HSI 16 MHz → PLL)의 오차는 ±1 % 이내로 규정되어 있어 이 차이를 설명하기에 충분하다.

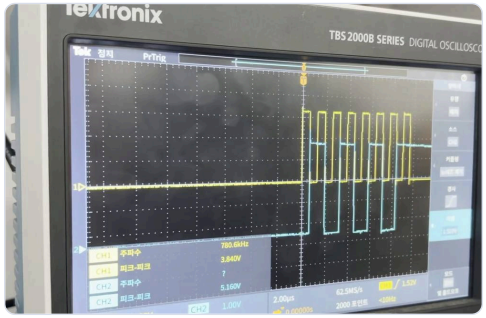


그림 1. 0x55 — 상승 에지마다 0 1 0 1 0 1 0 1, 780.6 kHz

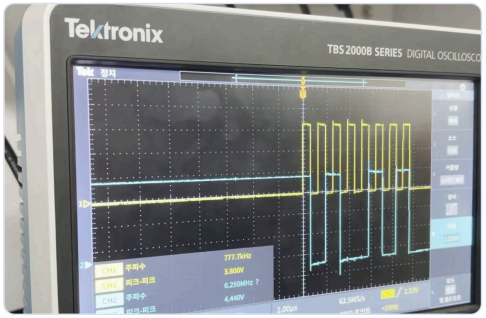


그림 2. 0xA5 — 1 0 1 0 0 1 0 1, 가운데 b4·b3가 연속 0이라 MOSI가 2주기 동안 LOW

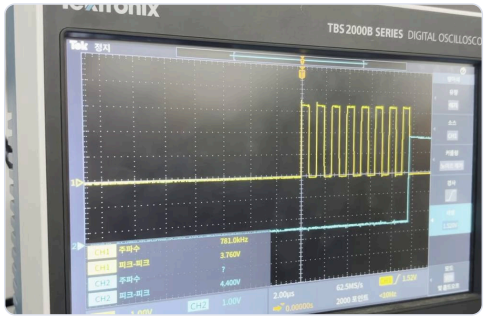


그림 3. 0x00 — MOSI는 LOW 그대로, SCK만 8번 (781.0 kHz)

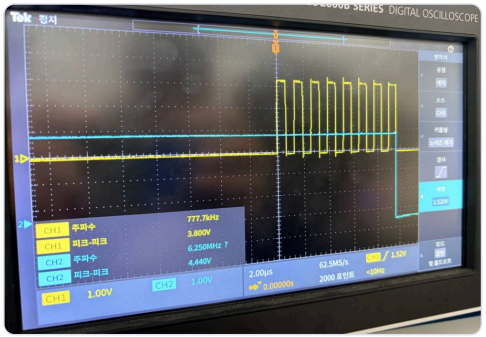


그림 4. 0xFF — MOSI가 8비트 내내 HIGH, 마지막 하강 에지 뒤 LOW로 돌아감 (777.7 kHz)

과제8 영상 (7.6 s) · Single로 바이트를 하나씩 잡은 화면

영상: LAB7/evidence/videos/LAB7-01_과제8_MCU송신파형.mp4 (GitHub에서 열기 ↗)

⑥ 문제 해결 과정 — 파형이 안 잡힘

처음 약 30분 동안 SCK(CH1)가 평평하게만 보였다. MCU의 Serial Monitor에는 `TX=55, result=0`이 나오고 있었으므로 원인을 오실로스코프와 연결 쪽에서 좁혔다.

단계	증상 · 확인한 것	원인 · 조치
1	**Autoset**을 누르자 1 s/칸 · 롤 모드 · 125 S/s로 바뀜. CH2(MOSI)는 1초 간격 느린 사각파, CH1(SCK)은 평평	1초에 한 번 10 µs만 나오는 신호는 125 S/s로는 찍히지 않는다. MOSI가 느린 사각파로 보인 것은 전송 사이에 마지막 비트 값을 유지하기 때문. → Autoset 대신 2 µs/칸으로 직접 설정 (그림 5)
2	2 µs/칸 · Normal 트리거로 바꾸자 화면 위에 "트리거?"만 뜨고 아무것도 안 그림	CH1에 1.52 V를 넘는 에지가 전혀 들어오지 않는다는 뜻 → 연결 확인
3	오실로스코프 Probe Comp 단자(5 V · 1 kHz)에 CH1 프로브 연결	5 V · 998 Hz 사각파가 정상으로 나옴 → 프로브와 채널은 정상
4	VS Code Serial Monitor에 `Closed serial port COM3 due to disconnection`, STM32Cube 장치 목록 "No device detected"	**NUCLEO USB 케이블이 빠져 보드 전원이 꺼져 있었다** (사진을 찍으려고 보드를 들다가 빠진 것으로 보임). 다시 꽂자 TX 출력 재개
5	Autoset이 다시 눌려 CH1이 AC 결합(∿)으로 바뀌어 있음	CH1 결합 DC, 트리거 CH1 · 상승 · 1.52 V · Normal, 2 µs/칸 → Single로 펄스 8개 포착

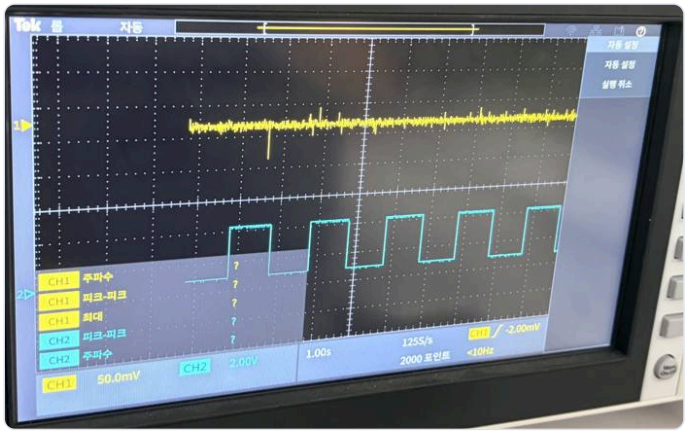


그림 5. Autoset 직후 화면 — 1.00 s/칸 · 125 S/s 롤 모드. CH2(파랑)의 느린 사각파는 바이트 사이에 MOSI가 유지한 마지막 비트이고, SCK 펄스는 보이지 않는다.

2. STM32_3 과제10 — MCU → FPGA 수신

자기 조 MCU(마스터)가 보낸 SPI 바이트를 직접 만든 FPGA 슬레이브 회로가 받아, FPGA USB-UART(F6)로 PC에 "RX=55"처럼 출력하고 MCU 송신값과 비교한다.

① 목적

FPGA가 자기 50 MHz 박자로, 남이 치는 SCK 박자를 알아듣는지 확인한다. 회로 설명(2단 동기화 → 상승에지 검출 → 시프트 레지스터 + 비트 카운터 → CS로 프레임 정리)은 실험 전 보고서 3장.

② Vivado 시뮬레이션

XSim 테스트벤치는 MCU 모델로 16바이트(덜 받은 프레임, 4바이트 프레임, CS HIGH 중 SCK 잡음, 1 MHz · 100 kHz 포함)를 보내고 PC로 나가는 UART 글자를 해독해 비교했다. 결과: `PASS spi_rx_top: 16 bytes, 14 lines`.

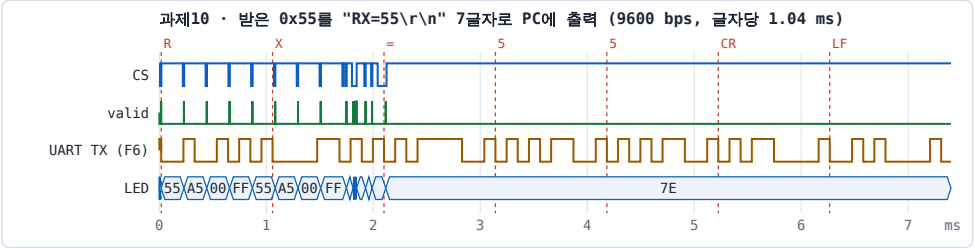
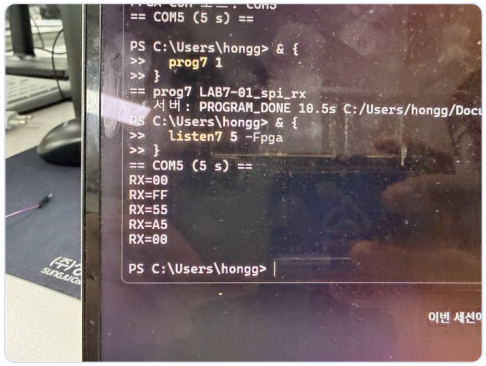


그림 S2. XSim — 받은 0x55를 F6로 "RX=55\r\n" 7글자로 출력 (9600 bps, 7.29 ms)

③ 보드 검증

비트스트림 `LAB7-01_spi_rx.bit`를 다시 빌드 없이 올리고(PROGRAM_DONE 10.5 s), NUCLEO D3 · D11 · D10을 J10 1 · 3 · 5번에, GND끼리 연결했다. MCU는 과제8 코드(55 → A5 → 00 → FF) 그대로다.

MCU 송신 (1초마다)	FPGA → PC (COM5)	FPGA LED (LED1 = b7)	판정
… 00 → FF → 55 → A5 → 00	`RX=00` `RX=FF` `RX=55` `RX=A5` `RX=00`	1초마다 같은 값으로 바뀜 (0x55 = ○●○●○●○●)	**순서 · 값 일치 ✔, `RX=?` (덜 받은 프레임) 0회**



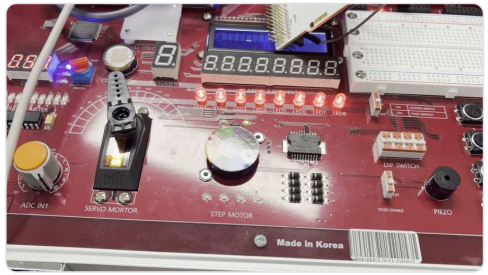


그림 7. FPGA LED 8개가 받은 바이트를 표시

그림 6. FPGA USB-UART(COM5) 5초 기록 — MCU가 보낸 순서 그대로 RX=00 · FF · 55 · A5 · 00

과제10 영상 (27.2 s) · PowerShell의 RX 출력과 FPGA LED

영상: LAB7/evidence/videos/LAB7-02_과제10_MCU-FPGA수신.mp4 (GitHub에서 열기 ↗)

④ 문제 해결

처음 5초 동안 COM5에 아무 글자도 오지 않았다. FPGA는 CS가 한 번만 내려갔다 올라와도 `RX=…` 나 `RX=?` 를 보내게 만들었으므로 "SPI 선이 FPGA에 아예 닿지 않음"으로 판단했고, 실제로 과제8에서 오실로스코프에만 물려 있던 선을 아직 J10으로 옮기지 않은 상태였다. 선을 옮기자 바로 정상 출력됐다.

3. STM32_3 과제11 — FPGA → MCU 송신

직접 만든 FPGA 마스터 회로가 1초마다 DIP 스위치 값을 SPI로 보내고(SCK ≤ 1 MHz), 슬레이브로 바꾼 MCU가 받아 VCP로 출력한다.

① 목적

이번에는 FPGA가 박자를 친다. 첫 비트는 CS를 내리면서 미리 준비하고, 이후 비트는 SCK 하강 에지에서 바꾸는 규칙을 실제 파형으로 확인한다. 설계값은 $f_{SCK} = 50\ \mathrm{MHz}/(2 \times 50) = 500\ \mathrm{kHz}$, CS LOW 17 µs.

② 순서 (출력끼리 부딪치지 않게)

1. MCU에 수신 코드(`task9_rx`)를 먼저 굽는다 → PB3 · PA7이 입력으로 바뀜.
2. CS 선의 MCU 쪽 끝을 D10(PB6) → **A2(PA4, NSS)**로 옮긴다.
3. 그다음 FPGA에 `LAB7-02_spi_tx.bit` 를 올린다(PROGRAM_DONE 10.5 s).

③ Vivado 시뮬레이션

TB의 MCU 슬레이브 모델이 모드 0 규칙(a ~ h)을 검사했다: `SCK 주기 2000.0 ns → 500.0 kHz` , `CS → 첫 상승 1000.0 ns` , `CS LOW 17.00 us` , 6프레임 `PASS` . 그림 8의 실측 파형과 비교할 기준이다.

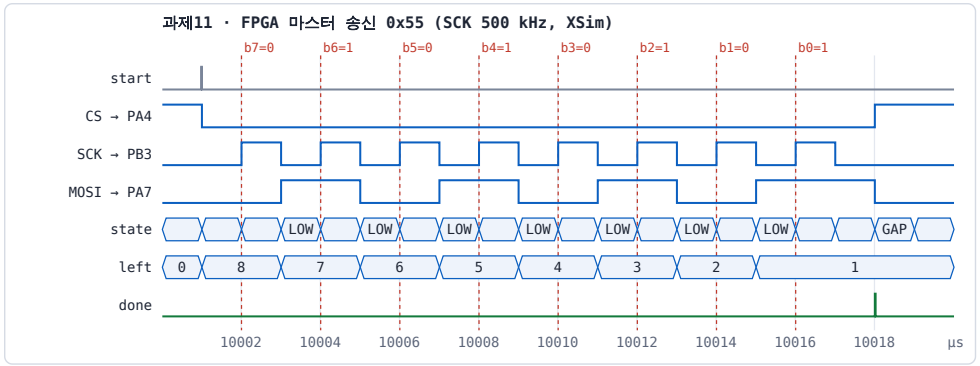


그림 S3. XSim — FPGA 마스터 송신 0x55. CS가 내려갈 때 b7이 이미 나와 있고 MOSI는 SCK 하강 에지에서만 바뀐다.

④ 보드 검증

확인	기대	관찰	판정
SCK 주파수	500 kHz	**502.6 · 501.5 · 498.7 · 501.2 kHz** (Single 4회)	**±0.5 % ✔**
한 프레임 SCK 펄스	8개	8개	✔
첫 비트 준비 (0xFF 송신 중)	CS가 내려갈 때 MOSI = b7(1)이 이미 나와 있고, 첫 SCK 상승은 1 µs 뒤	MOSI가 첫 SCK보다 먼저 HIGH가 되어 8비트 동안 유지, 마지막 비트 뒤 LOW (그림 8)	✔
MCU 수신 (DIP = 0x55)	`RX=55` 1초마다	10초 동안 `RX=55` 10번	✔
스위치를 바꾸는 중	바꾼 값이 다음 프레임에 반영	켜는 도중 `RX=00` → `40` → `48` (SW2 → SW2+SW5)	**실시간 반영 ✔**



그림 8. FPGA 송신 파형 (4 µs/칸, CH1 = SCK 502.6 kHz, CH2 = MOSI). 0xFF를 보내는 중이라 MOSI가 첫 SCK 상승 전에 올라가 8비트 동안 HIGH. 파란 선의 작은 뾰족한 모양은 SCK 에지가 옆 선에 유도된 것(크로스토크)으로, 상승 에지 순간의 값에는 영향이 없다.

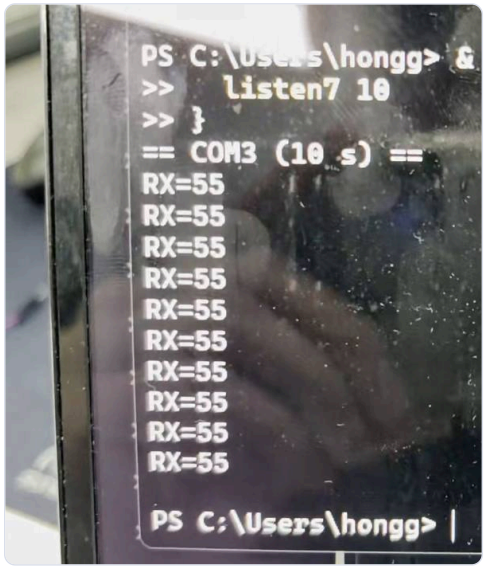


그림 9. MCU(COM3) 10초 기록 — RX=55 × 10

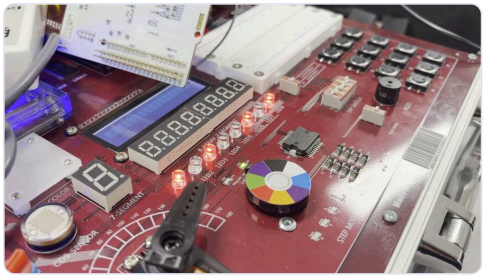


그림 10. DIP 스위치로 보낼 값을 바꾸는 장면, LED는 보낸 값

과제11 영상 ① (32.4 s) · DIP 스위치 · LED · MCU 출력

영상: LAB7/evidence/videos/LAB7-03_과제11_FPGA-MCU송신_a.mp4 (GitHub에서 열기 ↗)

과제11 영상 ② (6.5 s) · FPGA SPI 파형 (500 kHz)

영상: LAB7/evidence/videos/LAB7-03_과제11_FPGA-MCU송신_b.mp4 (GitHub에서 열기 ↗)

4. STM32_3 과제9 — MCU ↔ MCU (C조 – D조)

두 조의 MCU를 연결해 한쪽이 보내고 다른 쪽이 받는다. 받는 쪽 VCP 출력이 보낸 값과 같은지 확인하고 역할을 바꾼다.

① 연결과 순서

	마스터 (보냄)	슬레이브 (받음)	CS 선
1차	C조 (`task8_cycle`)	D조 (매뉴얼 수신 코드)	C조 D10(PB6) → D조 A2(PA4)
2차 (교대)	D조	C조 (`task9_rx`)	D조 D10 → C조 A2 (선을 옮김)

SCK(D3 ↔ D3), MOSI(D11 ↔ D11), GND는 그대로 두었다. 과제11에서 J10에 꽂았던 선은 먼저 FPGA 쪽에서 뽑았다(FPGA 송신 출력과 MCU 출력이 부딪치지 않게). 받는 쪽을 먼저 실행해 `RX timeout` 대기 상태로 두고 보내는 쪽을 리셋했다.

② 보드 검증

방향	보낸 쪽 출력	받은 쪽 출력	판정
C조 → D조	`TX=55 / A5 / 00 / FF, result=0` (COM3)	D조 COM7: `RX=55 A5 00 FF` 반복	✔
D조 → C조	D조 송신 (같은 4값 순환)	우리 COM3: `RX=55 A5 00 FF` 반복	✔

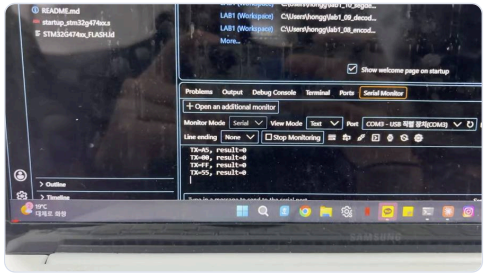


그림 11. 1차 — 우리 MCU의 송신 기록 (TX=..., result=0)

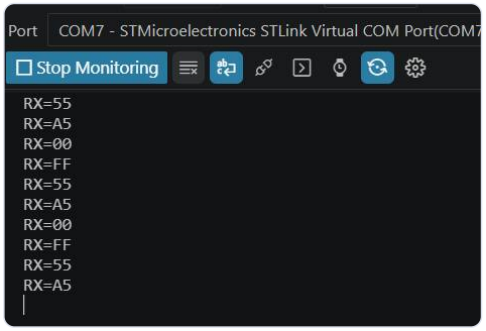


그림 12. 1차 — D조 MCU(COM7)의 수신 기록

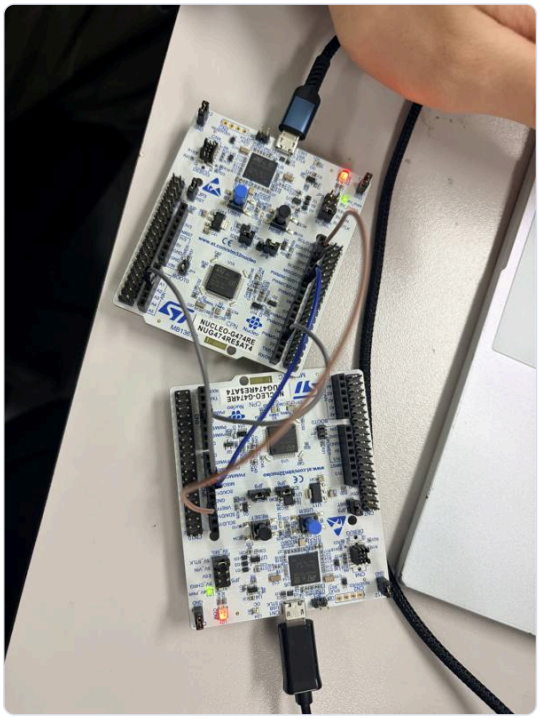


그림 13. 두 NUCLEO 연결 (SCK · MOSI · CS · GND)

과제9 영상 ① (8.1 s) · 1차 C조 → D조

영상: LAB7/evidence/videos/LAB7-04_과제9_MCU-MCU_a.mp4 (GitHub에서 열기 ↗)

과제9 영상 ② (4.6 s) · 2차 D조 → C조

영상: LAB7/evidence/videos/LAB7-04_과제9_MCU-MCU_b.mp4 (GitHub에서 열기 ↗)

5. STM32_3 과제12 — FPGA ↔ FPGA (C조 – D조)

한쪽은 과제11의 송신 회로, 다른 쪽은 과제10의 수신 회로로 FPGA끼리 SPI 통신하고, 받는 FPGA의 UART로 확인한 뒤 역할을 바꾼다.

① 연결

우리 J10 1 · 3 · 5번(SCK · MOSI · CS)과 GND를 D조 FPGA의 확장 포트에 연결했다. 역할을 바꿀 때는 **비트스트림만** 바꾸고(`LAB7-02_spi_tx` ↔ `LAB7-01_spi_rx`) 선은 그대로 두었다. 두 회로가 같은 핀을 입력/출력으로 쓰도록 미리 맞춰 둔 덕분이다. NUCLEO는 쓰지 않으므로 빼고, USB 두 개를 JTAG과 FTDI에 썼다.

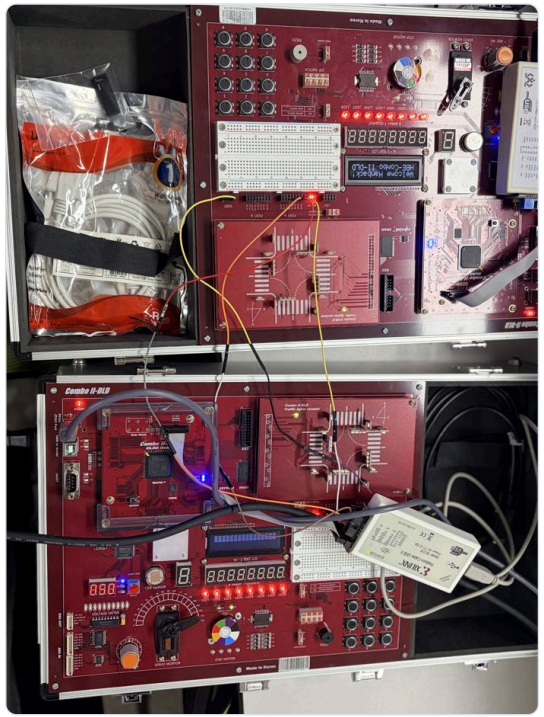


그림 14. FPGA 두 대 연결 (위 D조, 아래 C조). PORT A 헤더끼리 SCK · MOSI · CS · GND

② Vivado 시뮬레이션

두 보드를 한 TB에 넣고 클록을 일부러 어긋나게(A +100 ppm, B −100 ppm) 하고 선 지연 3 ns를 넣었다. 결과 `PASS FPGA->FPGA: 5 frames, B 수신 = A 송신` .

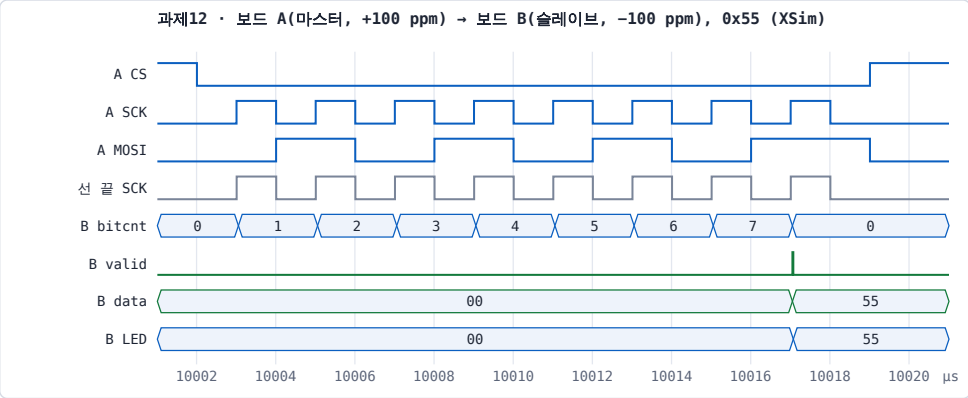


그림 S4. XSim — 보드 A 송신 0x55를 보드 B가 자기 클록으로 동기화해 받는 모습

③ 1차 — C조 송신 → D조 수신

D조 수신 회로는 받은 바이트를 **그대로** UART로 보낸다. 그래서 0x55가 Text 모드에서 ASCII 문자 `U` 로 보였다(ASCII 0x55 = U). View Mode를 Hex로 바꿔 `55` 로 확인했다.

우리 DIP 스위치	보낸 값	D조 Serial Monitor (Hex)	판정
SW2 · 4 · 6 · 8 ON	0x55	`55 55 55 …` (1초마다 하나)	✔
0x55 → 0xA5 → 0x00으로 바꾸는 중	중간값 포함	`55 … 95 b5 a5 a5 … a9 80 00 00`	✔ (아래 설명)

중간에 보인 `95 · b5 · a9 · 80` 은 오류가 아니다. 스위치를 한 칸씩 바꾸는 동안 1초마다 그 순간의 값이 전송된 것이다. 예를 들어 0x55(`0101 0101`)에서 0xA5(`1010 0101`)로 갈 때 SW1을 켜면 `1101 0101` = 0xD5, SW3까지 켜면 `1111 0101` = 0xF5처럼 비트가 하나씩 바뀐다. 관찰된 0xB5(`1011 0101`)와 0x95(`1001 0101`)도 이런 식으로 55와 A5 사이를 오가는 값이고, 0xA9 · 0x80은 A5에서 00으로 끄는 중의 값이다. 송신기가 매 프레임 시작 순간의 스위치 값을 붙잡아 보내도록 설계했으므로, 한 프레임 안에서 비트가 섞이지는 않는다.

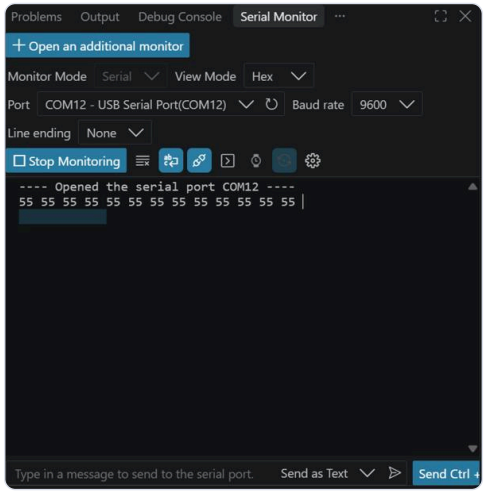


그림 15. D조 수신 (Hex) — 55가 1초마다

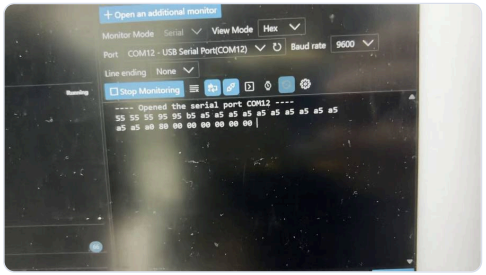


그림 16. 스위치를 바꾸는 동안의 D조 수신 — 중간값 뒤 A5, 00으로 정착

④ 2차 — D조 송신 → C조 수신

단계	우리 COM5 (FTDI)	원인 · 조치
처음	`18 33 80 66 f3 …` `98 e6 98 e6 98 1e …` — 의미 없는 바이트가 계속 옴 (그림 17)	**CLOCK SELECT 다이얼이 50 MHz가 아니라 그 절반 위치**였다. UART 송신기의 비트 폭은 5208클록으로 고정되어 있어, 클록이 25 MHz면 9600 → 약 4800 bps로 느려진다. "RX=55\r\n"을 절반 속도로 보내고 9600으로 읽는 경우를 계산하면 `… 98 e6 98 e6 98 1e …` 가 나와 관찰과 같은 묶음이 반복된다.
다이얼 F로	`RX=FF RX=55 RX=A5 RX=00` 반복, LED 같은 값 (그림 18)	**D조가 보낸 값(55 → A5 → 00 → FF 순환)과 일치 ✔**

흥미로운 점은 비트스트림을 다시 올리지 않고 다이얼만 돌려 바로 정상이 됐다는 것이다. SPI 수신부는 클록이 절반이어도 SCK 반주기(1 µs) 동안 25번 샘플링하므로 바이트는 맞게 받고 있었고, PC로 내보내는 UART 쪽만 속도가 어긋났던 것이다(고찰 2).

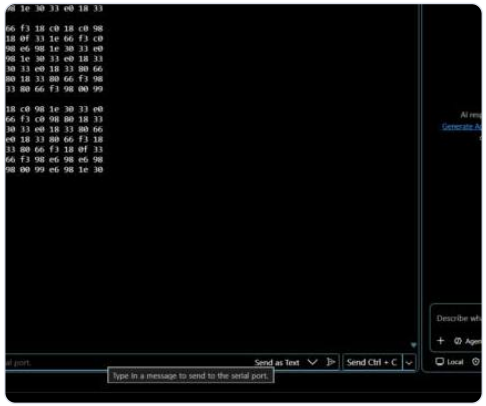


그림 17. 클록이 절반일 때 — 9600 bps로 읽은 깨진 바이트

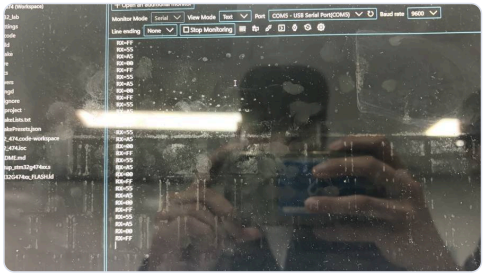


그림 18. 다이얼을 F(50 MHz)로 고친 뒤 — RX=FF · 55 · A5 · 00

과제12 영상 ① (11.9 s) · 2차 D조 → C조, 우리 Serial Monitor

영상: LAB7/evidence/videos/LAB7-05_과제12_FPGA-FPGA_a.mp4 (GitHub에서 열기 ↗)

과제12 영상 ② (28.0 s) · 1차 C조 → D조, 스위치 변경과 D조 Hex 출력

영상: LAB7/evidence/videos/LAB7-05_과제12_FPGA-FPGA_b.mp4 (GitHub에서 열기 ↗)

6. 교재 핀 대조 · 보드 빌드 결과

① 교재 p.73 확장 포트 대조

신호	방향 (수신 / 송신 회로)	우리 XDC	교재 p.73	J10 핀	결과
SCK	입력 / 출력	F8	BDIO_00 = F8	1	**일치**
MOSI	입력 / 출력	E8	BDIO_01 = E8	3	**일치**
CS	입력 / 출력	A7	BDIO_02 = A7	5	**일치**
PORT A ENABLE	—	—	DIP SW5의 2번 → U19 BE (활성 LOW, 4.7 kΩ 풀다운)	—	**OFF = 연결**

회로도에서 BDIO_00 ~ 07은 100 Ω 저항과 버스 스위치 U19(PI3C3245)를 지나 J10의 홀수 핀 1 · 3 · … · 15에 차례로 나온다. 그래서 SPI 세 선은 J10 한 줄에 이웃한 1 · 3 · 5번이 된다. 나머지 핀(클록 B6, 리셋 K4, LED, DIP, UART F6)은 LAB1 ~ 6에서 보드로 검증한 값을 그대로 썼다.

② 보드 빌드 (이 노트북에서 다시 빌드, 실험 전과 같음)

회로	Slice LUT	Slice Register	Bonded IOB	WNS	WHS	사용 과제
spi_rx_top (수신)	160	141	14	+14.907 ns	+0.036 ns	과제10 · 12
spi_tx_top (송신)	207	202	22	+14.323 ns	+0.082 ns	과제11 · 12

③ 시뮬레이션 대비 실측

항목	시뮬레이션 (실험 전)	보드 실측
MCU SCK	781.25 kHz (모델)	777.7 ~ 781.0 kHz
FPGA SCK	500.0 kHz (주기 2000 ns)	498.7 ~ 502.6 kHz
FPGA CS → 첫 SCK 상승	1.00 µs	그림 8에서 약 ¼칸(4 µs/칸) ≈ 1 µs
FPGA → PC 출력 형식	"RX=55\r\n"	같음
덜 받은 프레임	TB에서 일부러 만들어 "RX=?" 확인	실험 중 0회
FPGA ↔ FPGA	클록 ±100 ppm, 선 지연 3 ns에서 PASS	양방향 정상 (클록 다이얼을 맞춘 뒤)

7. 고찰

1. **Autoset은 드문 신호를 숨긴다.** SPI 한 바이트는 1초 중 10 µs, 즉 0.001 %의 시간에만 존재한다. Autoset은 화면에 "늘 보이는" 성분(바이트 사이에 MOSI가 유지하는 레벨)을 기준으로 1 s/칸 롤 모드를 골랐고, 125 S/s에서는 SCK가 아예 찍히지 않았다. 드문 짧은 사건은 **예상 시간축을 직접 계산**(10 µs → 2 µs/칸)하고 **Normal 트리거 + Single**로 기다려야 잡힌다. LAB6에서 정리한 "시간축 = 신호 주기 ÷ 10칸" 규칙을 이번엔 "사건 길이 ÷ 5칸"으로 적용한 셈이다.
2. **SPI는 받는 쪽 클록에 강하고, UART는 약하다.** 과제12 2차에서 FPGA 클록이 절반이 되었는데도 SPI 수신은 맞았고 UART 출력만 깨졌다. SPI는 마스터가 SCK로 박자를 직접 보내므로 받는 쪽은 에지만 놓치지 않으면 되고(25 MHz로도 반주기당 25번 샘플), UART는 양쪽이 각자 시계로 비트 폭을 재므로 한쪽 클록이 바뀌면 바로 어긋난다. 같은 회로 안에서 두 방식의 차이가 그대로 드러났다. 깨진 바이트를 "클록 절반" 가정으로 계산해 화면과 같은 묶음이 나오는지 비교한 것이 원인을 빠르게 확정하는 데 도움이 됐다.
3. **규칙을 먼저 맞추면 남이 만든 회로와도 바로 통한다.** D조 회로는 출력 형식(바이트 그대로)이 우리(글자 "RX=55")와 달랐지만, 모드 0 · MSB 먼저 · CS 프레임이라는 규칙만 같아서 별도 조정 없이 첫 연결에서 통신됐다. 역할 교대도 핀을 같게 설계해 선은 그대로 두고 비트스트림만 바꿔 끝냈다.
4. **문제의 대부분은 회로가 아니라 연결이었다.** 이번 실험에서 막힌 곳은 USB 케이블 빠짐, 선을 옮기지 않음, 오실로스코프 설정, 클록 다이얼이었다. 그래서 확인 순서를 "출력 쪽 로그(Serial Monitor) → 기준 신호(Probe Comp) → 보드 전원 · 연결 → 설정"으로 가져가니 원인이 빨리 좁혀졌다. 특히 VS Code의 "disconnection" 메시지가 30분짜리 오실로스코프 문제의 실제 원인을 알려 주었다.
5. **result = 0과 RX의 차이.** 과제8 내내 MCU는 `result=0`을 출력했지만, 그 시점에도 오실로스코프와 FPGA에는 신호가 없었다(USB가 빠지기 전후). 송신 함수의 성공은 "MCU 안에서 보내기를 마쳤다"일 뿐이라, 이번 실험의 모든 판정은 받는 쪽 출력(FPGA COM5, MCU COM3 · COM7)과 파형으로만 했다.
6. **기말 프로젝트로 이어지는 점.** 수신기는 CS 한 프레임 안의 여러 바이트를 번호와 함께 받고, 송신기는 바이트 수를 매개변수로 바꿀 수 있게 만들었다. 프로젝트의 FPGA–STM32 4바이트 레지스터 맵을 이 두 모듈로 바로 연결할 수 있다.

참고문헌

1. [1] D. M. Harris and S. L. Harris, *Digital Design and Computer Architecture*, 2nd ed. Waltham, MA, USA: Morgan Kaufmann, 2013. ISBN 978-0-12-394424-5. — Ch. 3.5.5 동기화기, Ch. 8.6.3 SPI와 UART.
2. [2] STMicroelectronics, *stm32g4xx-hal-driver*, GitHub, https://github.com/STMicroelectronics/stm32g4xx-hal-driver, 태그 v1.2.7. — `stm32g4xx_hal_spi.c`의 `HAL_SPI_Transmit` · `HAL_SPI_Receive`.

수업 자료(인용 출처 수에 넣지 않음): Glaysia/ece2_474 v1.1.0 및 Glaysia/ece2_26_2_2의 STM32_3 문서, 실습 교재 「Combo II DLD 장비를 이용한 디지털 논리 회로 설계」 p.73(확장 포트 회로도 · 핀 구성표), Tektronix TBS2000B 사용 설명(트리거 · 결합 설정).